Phases Make Failure Detectable: Phase-Structured LTL Runtime Monitoring of Robot Skills

[TODO: author block, affiliations, funding, and the RA-L copyright / accompanying-video footnotes]

***Abstract*—A runtime monitor for a robot skill is usually handed the skill's requirements as a flat set of temporal-logic formulas. We show this is not merely coarse but structurally deficient. The formula that states what a skill is *for* is a guarantee property; guarantee properties have no bad prefix; so a flat specification is monitorable for satisfaction but *not for violation*, and a stalled robot leaves it permanently inconclusive. Our claim is that decomposing a skill's execution into *phases* repairs this, and that the repair is the main reason to reach for temporal logic at all. Each phase carries a precondition checked on entry, an invariant checked every tick, a progress condition with a bounded violation budget, an exit condition, and a deadline in ticks. Bounding each obligation turns a guarantee property into a ladder of safety properties, each with bad prefixes of known length and therefore refutable on a finite trace; and because every guard belongs to a named phase and carries a fault category, a breach arrives already localised and already graded into an intervention. The phase ladder does not replace Linear Temporal Logic but runs alongside it: LTL carries the cross-cutting invariants and supplies the monotone, permanent verdict a supervisor can latch, while the ladder supplies the bounded obligations and the diagnosis LTL structurally cannot. We also observe that the phase structure is what *manufactures* the finite trace a finite-trace semantics presupposes — a deadline is what closes an episode — so the claim does not depend on which finite-trace semantics is adopted. Two further properties follow from the same decomposition rather than being designed for: the monitor never names an embodiment, so one specification runs on hardware, in simulation and on a recorded stream; and a phase ladder over a declared vocabulary can be authored from a free-language skill description and validated mechanically. We evaluate on humanoid navigation with a Unitree G1, ablating the phase ladder against the flat specification of the same mission. [TODO: headline result]**

***Index Terms*—Formal methods in robotics and automation, runtime verification, failure detection and recovery, task and motion planning, humanoid robots.**

I. Introduction

CONSIDER a humanoid executing a waypoint navigation mission. It is armed, it plans, it walks, and then it wedges itself against a lip its traversability estimate called flat. The legs keep cycling. The planner keeps reporting that it is following a path. The goal is never reached.

Ask what a runtime monitor should say. The requirement the skill was given is, in Linear Temporal Logic (LTL), something like

$$\mathbf{F}\big(\mathit{armed} \wedge \mathbf{F}(\mathit{planning} \wedge \mathbf{F}(\mathit{walking} \wedge \mathbf{F}\mathit{finished}))\big), \quad (1)$$

and this is the form such specifications actually take — the shipped specification for our platform contains exactly this formula. On the trace above, a monitor for (1) reports INCONCLUSIVE. It reports INCONCLUSIVE on the next tick, and on every tick after, for as long as anyone watches. It is not reporting late. It is reporting correctly and will never report anything else, because (1) is a guarantee (co-safety) property: at any finite point, *finished* could still become true later. Guarantee properties have no bad prefix [1], and a property with no bad prefix admits no finite counterexample [2].

This is the problem we address, and it is not a matter of degree. A flat temporal specification of a skill's goal is monitorable for satisfaction but not for violation, and the commonest robot failure — making no progress — is exactly the one it cannot refute.

A. Phases make the obligation bounded

Our claim is that the repair is to decompose a skill's execution into **phases**, and that this decomposition is what makes runtime monitoring of a skill work at all.

A phase is a named stage carrying five guards and two budgets: an *enter* condition, a *precondition* checked once on entry, an *invariant* checked every tick the phase is active, a *progress condition* with a bounded violation budget, an *exit condition*, and a *deadline* in ticks. Figure 1 shows our platform's mission expressed both ways, from one specification file.

The consequence is formal, not cosmetic. "Eventually finish" is a guarantee obligation with no bad prefix. "Leave this phase within k ticks" is *bounded*: a safety property with bad prefixes of length at most k, refutable on a finite trace. So is "the progress predicate does not fail on m consecutive ticks". Phase decomposition with deadlines performs a **guarantee-to-safety conversion**, one stage at a time, and that conversion is what turns an unrefutable stall into a detected one. On the wedged-humanoid trace the ladder reports a timeout fault at the phase deadline while the flat formula is still inconclusive. It does *not*, as shipped, report the earlier progress fault: the progress condition in our specification is itself written over the planner's status, so a planner that keeps reporting `following` keeps it satisfied. Only the deadline, which reads nothing, catches this case today; the progress budget catches it once progress is derived from odometry (Section VIII).

B. Phases also localise and grade

Detectability is the formal argument; three further properties come from the same structure.

A breach names a stage and a guard. Phase faults are named `phase:`⟨`name`⟩`:`⟨`invariant | timeout | progress | precondition`⟩. An operator is not told that a conjunction failed but that the tracking phase stopped progressing. "Where in the skill is it" is the first question anyone asks of a running robot, and a product automaton's state is not an answer to it.

Flat LTL (as shipped)	Phase ladder (as shipped)
One formula: $\mathbf{F}(armed \wedge \mathbf{F}(planning \wedge \mathbf{F}(walking \wedge \mathbf{F}finished)))$	Three phases, each with guards and budgets: PLANNING: deadline 30, budget 5
On a stalled trace: INCONCLUSIVE, forever.	
Diagnosis: none. The formula names the mission, not the stage.	TRACKING: deadline 120, budget 5
	CONFIRM: deadline 10
On a stalled trace: FAILURE at the deadline; at the budget if progress is sensed independently of the planner.	
Diagnosis: `phase:tracking:timeout`, category TIMEOUT → REPLAN.	

Fig. 1. The same navigation mission, encoded two ways in the same specification file. The left column is monitorable for satisfaction but not for violation; the right is refutable, localises a breach to a stage and a guard, and grades it into a response. **[TODO: replace with a drawn figure: the two encodings plus an annotated stall trace showing the tick at which each reports.]**

A breach carries a category, and the category picks the response. Guards declare fault categories that grade into an intervention ladder: a safety breach halts, a timeout or progress breach requests a replan. Severity travels with the fault instead of being re-derived downstream.

The same proposition can mean different things at different stages. *walking* false is unremarkable while planning and is a progress violation while tracking. A flat specification expresses this only by encoding the stage into the formula — reinventing phases inside LTL, at the cost of automaton states that mean nothing to an operator.

C. Why LTL, then, and not phases alone

Phases supply bounded obligations and diagnosis. They do not supply a formal temporal semantics, they do not express requirements holding *across* stages, and a hand-rolled stage machine cannot promise a verdict will not be retracted. LTL supplies all three. The invariants that matter most on a humanoid — never let an obstacle within $0.25\,\mathrm{m}$, never stop being upright — hold in every phase and between phases, and are naturally $\mathbf{G}\neg collision_risk$ and $\mathbf{G}\,upright$. Compiled to automata they also give what a supervisor needs and a stage machine cannot: violation is *permanent*, because the automaton enters a non-accepting trap state, so a latched safety response never risks the verdict being withdrawn.

The two layers are therefore not alternatives, and the choice of LTL is not incidental to the phase claim — it is the half of the design that makes the other half safe. Section II sets out how LTL works, which semantics we monitor it under, and why we chose it over the other ways a skill's requirements could be declared; Section III covers what others have built; Section IV is the phase argument.

D. Two consequences we did not design for

Because every guard is written only over atomic propositions, and each proposition's grounding in sensors is declared separately, nothing in the phase structure names an embodiment: one specification runs against hardware, a simulator and a recorded stream, and the claim is checkable rather than asserted. Because a phase ladder over a declared vocabulary is a small declarative artifact, it can be generated from a free-language skill description and then validated mechanically — every sensor key a guard references must be declared, and every formula must compile before a monitor will adopt it. Neither is the point of the paper, but both are evidence the decomposition cuts along a real joint.

E. Contributions

1) **The detectability argument**: flat temporal specifications of robot skills are not refutable, this is why stalls go undetected, and bounding each stage's obligation is what repairs it, with a proved horizon on time to verdict (Section IV).
2) **A phase structure for monitoring**, with five guard kinds and two budgets, fault names that localise, and categories that grade — run *alongside* LTL automata rather than in place of them (Sections IV, VI).
3) **The observation that phases manufacture finiteness**, which makes the claim independent of the choice between finite-trace semantics (Section II-B).
4) **Three-valued propositions with freeze-on-unknown, scoped per phase**, plus a per-tick DECIDED / UNDECIDED axis distinct from the LTL verdict (Section IV-E).
5) **An ablation on humanoid navigation** with a Unitree G1, running flat and phase-structured encodings of one mission over the same recorded episodes. **[TODO: result]**

II. Background

A. LTL in brief

LTL [3] extends propositional logic over a finite set AP of atomic propositions with operators over discrete time. A model is an infinite word $w = w_0 w_1 \ldots$ with $w_i \subseteq AP$ the propositions true at position i; w^i denotes the suffix from i. Writing $\models$ for satisfaction, the operators we use are

$$
\begin{aligned}
w \models \mathbf{X}\varphi &\;\Leftrightarrow\; w^1 \models \varphi && \text{(next)}\\
w \models \mathbf{G}\varphi &\;\Leftrightarrow\; \forall i \geq 0,\ w^i \models \varphi && \text{(always)}\\
w \models \mathbf{F}\varphi &\;\Leftrightarrow\; \exists i \geq 0,\ w^i \models \varphi && \text{(eventually)}\\
w \models \varphi\,\mathbf{U}\,\psi &\;\Leftrightarrow\; \exists i \geq 0,\ w^i \models \psi \wedge && \\
&\qquad\quad \forall j < i,\ w^j \models \varphi && \text{(until)}
\end{aligned}
$$

with the usual Boolean connectives. A safety requirement is written $\mathbf{G}\neg collision_risk$, a liveness requirement $\mathbf{G}(armed \rightarrow \mathbf{F}\,finished)$. In our system one position is one *tick* (Section V), so $\mathbf{X}$ means exactly one tick later.

Monitoring is automata-theoretic [4]. A formula is compiled into an ω-automaton over the alphabet 2^{AP} whose

runs accept exactly the words satisfying it; the monitor then tracks the automaton's state as the observed word arrives. We use Spot [5] to obtain deterministic, complete, state-based Büchi automata with transition labels as BDDs over AP, so one step is a single BDD evaluation against the current proposition assignment plus a state assignment — independent of how long the episode has run. The cost does grow with the number of propositions and with the automaton, which can be exponential in the formula; our formulas are small. Determinism is requested of Spot rather than guaranteed, since not every LTL formula has a deterministic Büchi automaton, so the specification contract should — and does not yet — reject a formula whose automaton is not deterministic. Completeness matters operationally rather than theoretically: a complete automaton has a successor for every assignment, so an unexpected observation never leaves the monitor undefined.

Two structural facts carry weight later. First, *violation is permanent*: a violated formula's automaton sits in a non-accepting trap state with a self-loop on $\top$, so the verdict is monotone. Second, *the formula layer mentions only AP*, which are opaque symbols: $\mathbf{G}\neg collision_risk$ says nothing about a depth camera. Everything about how a proposition is measured lives elsewhere.

In the safety/liveness hierarchy [1] two classes matter here. A *safety* property is violated by a finite bad prefix — one tick in which the proposition is false refutes $\mathbf{G}\neg collision_risk$. A *guarantee* (co-safety) property, such as (1), has no bad prefix: for any finite trace some extension satisfies it.

B. Monitoring a finite observation: which semantics

LTL is interpreted over infinite words, and a monitor sees a finite prefix. Three semantics answer this differently, and the choice is not cosmetic, so we state ours.

LTL$_3$ [2] evaluates a finite prefix u to one of three verdicts: $\top$ if *every* infinite continuation of u satisfies φ, $\bot$ if *no* continuation does, and ? otherwise — the prefix can still be extended either way. Note what this does and does not assume: LTL$_3$ quantifies over infinite continuations, so it is the semantics of a finite *observation of an open-ended run*, not the semantics of a trace believed to be complete.

LTL$_f$ [6] instead reinterprets the logic over finite traces, where the trace *is* the word and has a last position. There $\mathbf{F}\,finished$ is simply *false* on a trace in which *finished* never holds.

RV-LTL [7] refines LTL$_3$'s single ? into two values by additionally evaluating the prefix as if it were complete, yielding a four-valued verdict that distinguishes "presumably true" from "presumably false".

We monitor under **LTL$_3$** for the verdict that matters, violation. The correspondence with the monitor's three statuses is exact on one side only, and we state it rather than claim more. VIOLATED is LTL$_3$'s $\bot$: the automaton is in a non-accepting sink, so no continuation satisfies the formula, and the verdict is permanent. It is sound; it is complete only where Spot has collapsed every rejecting region to a single sink, which holds for the safety formulas we use and is not checked in general. ACCEPTED is *not* LTL$_3$'s $\top$. It reports that the current state is accepting — for $\mathbf{G}\neg collision_risk$ from the first tick, although the formula can still be violated — and it can revert, so it is closer to RV-LTL's "presumably true" than to a definitive verdict. Nothing downstream acts on it. The reason LTL$_3$ is the right reading for violation is that at tick k the monitor genuinely does not know whether the episode continues, and must publish a verdict anyway; LTL$_3$ is the semantics of precisely that situation. It is also why INCONCLUSIVE is the *normal, healthy* state of every liveness obligation not yet discharged, rather than an error.

One point of terminology, because it is easy to overclaim. Under the standard definition — φ is monitorable after u if some extension of u yields a definitive verdict — $\mathbf{F}p$ *is* monitorable, and the monitorable class is strictly larger than safety and co-safety together [2]. What (1) lacks is not monitorability but a *bad prefix*: it is monitorable for satisfaction and not for violation. We use that phrasing throughout.

Why not LTL$_f$ as the online semantics. It is tempting, because under LTL$_f$ the stalled run of Section IV does get a definitive *false*. But that verdict is available only at the last position, and LTL$_f$ cannot say *when the trace ends*. Something outside the logic must declare a tick to be the last one, and before that declaration an online LTL$_f$ monitor has nothing to report. In our system the thing that closes the episode is a phase deadline or a terminal condition — so the bounded obligation is doing the work either way. This is worth stating as a result rather than a caveat: *the phase structure manufactures the finite trace that a finite-trace semantics presupposes.* We therefore use LTL$_3$ online, per tick, and a finite-trace reading for the episode fold once the ladder has terminated. The phase claim is not an artifact of choosing LTL$_3$; adopting LTL$_f$ relocates the need for phases rather than removing it.

Why not RV-LTL. It is the sharpest alternative, since "presumably false" on a stalled prefix looks like the missing detection signal. Three reasons it is not. It is not monotone — the value flips back the moment the robot moves, so a supervisor has nothing to latch. It is *uninformative about progress*: $\mathbf{F}\,finished$ is presumably false on every prefix of a perfectly *successful* episode until the final tick, so it does not distinguish a healthy run from a stalled one. And it still never says when to give up. RV-LTL gives a richer per-tick readout; it does not give a failure decision.

C. Ways to declare what a skill must do

LTL is one of several ways to write down a skill's requirements. Table I compares them on the five properties a *monitor* needs, as opposed to the properties a planner or a controller needs.

Most rows fail on a single column and the table says why; two deserve extended comment, because they are the alternatives a reader will press hardest.

Behaviour trees and finite-state machines. These deserve a careful answer rather than a dismissal: they are how robot behaviour is actually structured, they are legible, reactive and modular, they carry condition nodes, and failure handling has been built on exactly that — verifying a tree's existing conditions and adding missing preconditions at runtime [16], [17].

TABLE I

WAYS TO DECLARE A SKILL'S REQUIREMENTS, JUDGED BY WHAT A RUNTIME *monitor* NEEDS. **T**EMPORAL: EXPRESSES ORDERING OR PERSISTENCE ACROSS THE EPISODE. **I**NDEPENDENT: IS NOT THE CONTROLLER, SO A MONITOR BUILT FROM IT CAN CATCH THE CONTROLLER BEING WRONG. **C**HECKABLE: A GENERATED INSTANCE CAN BE REJECTED MECHANICALLY, WITH NO HUMAN AND NO ROBOT. **P**ERMANENT: A VIOLATION VERDICT IS MONOTONE. **B**OUNDED: THE FORMALISM FORCES A PROGRESS OBLIGATION TO CARRY A HORIZON; "DEC." MARKS A BEHAVIOUR TREE'S TIMEOUT *decorator*, WHICH BOUNDS A NODE BUT IS THE CONTROLLER ABORTING ITSELF RATHER THAN A VERDICT.

formalism	T	I	C	P	B
free-language description	—	y	**no**	**no**	**no**
FSM / behaviour tree [8], [9]	impl.	**no**	part	**no**	dec.
action pre/post-cond. [10]	act.	y	y	**no**	**no**
reward machine [11]	y	y	y	**no**	**no**
propositional invariants	**no**	y	y	—	—
STL / MTL [12], [13]	metric	y	y	y	y
learned detector [14], [15]	impl.	**no**	**no**	**no**	**no**
LTL [3]	y	y	y	y	**no**
LTL + phase ladder (ours)	y	y	y	y	y

Our phase ladder is deliberately BT-shaped in its sequencing, and we note that a timeout *decorator* already bounds a node's duration, so the per-stage horizon of Section IV-B is not foreign to the formalism — the difference is that a decorator makes the controller abort itself, where ours produces a categorised verdict an independent supervisor acts on. Three things separate them, in descending order of how much they decided the matter.

First, a behaviour tree is an executable, not a declared property. Behaviour trees do have formal semantics, and a tree of condition nodes evaluated every tick denotes a state invariant, so this is not a claim that they lack trace semantics. It is that the property a tree enforces is implicit in its traversal and has to be extracted before one can ask which class it belongs to, whereas a formula *is* the property. The argument of this paper — that the goal obligation is a guarantee property and therefore unrefutable, and that bounding it makes it refutable — is made directly on the artifact in one case and on a derived model of it in the other.

Second, there is no permanence. A condition node is recomputed every tick and returns success, failure or running; nothing latches. One can add latching by hand, per node, but then the monotonicity a supervisor relies on is a property of hand-written bookkeeping rather than of the construction, and there is nothing to appeal to when it is wrong. An LTL automaton's trap state gives it for free (Section II-A).

Third, a generated tree is a program. Validating one means validating a program; validating a generated formula means compiling it and checking that the keys it names are declared (Section II-D). That difference is what makes authorship from a description safe, so it is not separable from the previous column.

The practical objection stands alongside the formal ones: in robotics a behaviour tree is usually *the controller*, and a monitor that is the controller cannot catch the controller being wrong. Our wedged humanoid is the case — the stack published `following` while the robot was stuck, so any monitor reading that state machine was blind to the fault structurally, not accidentally.

That leaves the two rows that satisfy almost everything. **STL and MTL** [12], [13] interpret temporal operators over real-valued dense-time signals with a quantitative robustness semantics, and — importantly for us — their metric operators *do* give bounded obligations. STL is therefore the honest main alternative, and we say why we did not take it: the robustness value is a scalar, so it answers how badly rather than which commitment, and which commitment is what the intervention ladder acts on; and quantitative semantics adds its own calibration burden, a choice of robustness definition and a normalisation across incommensurable units, on top of the thresholds we must calibrate regardless. We instead push the continuous part down into proposition grounding, where a threshold is one declared number in one place. Nothing in the phase structure precludes STL guards, and that is the obvious extension.

We chose **LTL** because it is the only row that is temporal, independent of the controller, mechanically checkable and permanent — and because the one column it fails, bounded obligations, is exactly the column the phase ladder fills. That is the shape of the paper's argument in one line: LTL supplies everything a monitor needs except a horizon, phases supply the horizon, and the combination is the last row of Table I.

D. Why author from a natural-language description

A specification has to come from somewhere, and there are three places. It can be **hand-authored** in the formal language, which is the status quo and the reason a platform with thirty skills has monitors for one or two: the cost scales with skills times embodiments, and it is paid by whoever can write temporal logic. It can be **learned** from labelled failure episodes, which works — $93\,\%$ phase accuracy from 52 labelled episodes [15], or $49\,000$ synthesised failure pairs [18] — at the cost of collecting and annotating those episodes per skill, and of binding the monitor to the embodiment through its training data. Or it can be **derived from a description of the skill**, which is the one artifact that already exists, because somebody had to say what the skill does before anyone built it.

The objection to the third option is that natural language is ambiguous, and it is. Our answer is not that the model is reliable. It is that **the model proposes and the contract checks**. Two gates stand between a generated specification and a running monitor, both mechanical, neither needing a human or a robot. The first runs inside the generation loop: every proposition a formula, guard or terminal condition uses must be declared, and every sensor key named by a grounding rule must be declared in the embodiment's schema; a proposal failing it is fed back to the model for repair. The second runs when a monitor loads the specification: every formula must compile under Spot [5], and a specification that does not is rejected with its problems published rather than adopted. Only the first gate is inside the repair loop today. A malformed proposal is therefore rejected rather than run, although an ambiguous description can still yield a well-formed specification that says the wrong thing, which no gate

sees — the problem the calibrated-abstention line addresses for translation itself [19], [20].

And the phase structure is what makes those gates strong. This is where the paper's two concerns meet. A phase ladder over a declared vocabulary is a far more constrained artifact than free-form temporal logic: a sequence of stages, each carrying a fixed, small set of guard kinds, each guard a Boolean expression over declared keys, plus two integers. There is correspondingly less room for a generated instance to be malformed in a way the gates cannot see. The same decomposition that makes failures refutable also makes the specification checkable, which we take as evidence it is cut along a real joint rather than as two independent conveniences.

What the gates cannot catch is whether a specification that compiles and type-checks says what the author *meant*. We mitigate rather than solve this: the generator also emits a prose description back from the specification it produced, so the human reviews English rather than LTL, and a mismatch is visible without reading a formula. The residual is real and is stated again in Section VIII. A wrong *threshold*, in particular, passes both gates and every review — which is why Section VII treats the bounds as an object of measurement rather than a detail.

III. Related Work

A. Runtime verification

Runtime verification checks one observed execution against a formal property [21], [22], canonically via the automaton construction of Section II-A. The semantics question of Section II-B is itself a subject in that community [2], [7], [6]. Our premise is a known property-class fact [1] applied to robot skills, where the goal formula is always of the guarantee shape.

B. Temporal logic for robot tasks, and hierarchy

Temporal logic entered robotics mainly through synthesis: construct a controller satisfying a specification by construction, or prove the task infeasible [23], [24]. That programme produces behaviour; we watch it, which removes the need for an environment model but adds the requirement that the monitor be able to *disagree* with the controller.

Closest in form to our ladder is the hierarchical-LTL line, which decomposes a flat formula into a task tree because "as task complexity increases, LTL formulas become lengthy" and unwieldy [25]. The motivation overlaps; the conclusion differs decisively. That decomposition exists to make *planning and allocation* tractable, and what it produces is still LTL, so each sub-formula inherits the property class of its parent and with it the absence of a bad prefix. Ours exists to make *monitoring* possible, and what it attaches to a stage — a deadline, a progress budget, a fault category — is exactly what a planning decomposition has no use for and does not carry.

C. Stage and phase structure in monitoring

Execution monitoring is an old problem — detect, during execution, that a plan is not going as intended [26] — and that it needs a notion of stage is increasingly recognised. This is the literature our claim must be measured against.

The sharpest comparison is work treating phase identification as a *learning* problem. RoboMonitor states the premise almost as we do — execution monitoring requires identifying the current execution phase, detecting failures and recognising completion — then trains a vision–language monitor to predict all three, reaching $93\,\%$ phase accuracy from 52 labelled episodes [15]. ConditionNET learns preconditions and effects and detects anomalies by comparing the expected motion phase with the predicted one [10]. We take the shared premise as support and part company on where the phase comes from. A learned phase estimate is itself a source of error, and its characteristic failure is instructive: RoboMonitor must actively suppress *spurious phase switching*, from $15.2\,\%$ to $5.0\,\%$, using temporal-consistency objectives. In our system the phase is not estimated. It is a state of a declared machine that advances only when a declared exit condition holds, so it cannot flicker between stages. The corresponding failure is different and not milder: a one-tick glitch in an exit condition advances the ladder irreversibly, which a minimum-dwell bound mitigates and does not remove. The uncertainty is pushed down into the proposition layer where it is explicit and three-valued (Section IV-E). The trade runs the other way too: a learned monitor needs no specification, and ours does.

Runtime verification has also been built around behaviour trees' execution context [17], which inherits both their hierarchy and, per Section II-C, their lack of independence from the controller.

Converging evidence arrives from outside robotics. For long-horizon LLM agents, DynSTEER argues the granularity point directly — whole-trajectory verification is too coarse to localise a failure, atomic-step scoring too noisy — and measures it: stage-wise evaluation improves discriminability by over $85\,\%$ against whole-trajectory evaluation, and permits early termination of unrecoverable runs [27]. That supports the granularity premise in a different domain; it is a result about discriminating between agents by evaluation score, not about detecting failures, and it is different in kind: those stages are compiled into a milestone graph and scored by LLM judges, with no formal guards, no deadline semantics and no claim about property classes.

D. Failure detection and reasoning with foundation models

The fastest-moving neighbouring literature detects failure with foundation models rather than from a specification. REFLECT summarises multisensory experience into a hierarchy and has an LLM explain the failure after the fact, contributing the RoboFail benchmark [28]. AHA frames failure reasoning as free-form language rather than binary classification and fine-tunes a VLM on procedurally perturbed demonstrations [18]; Guardian scales synthetic failure data further and verifies both proposed plans and subtask executions [29]. Closer still to a declarative monitor, DoReMi has an LLM emit per-skill *constraints* alongside the plan and a VLM check them every $0.2\,\mathrm{s}$ during execution [30], and Code-as-Monitor compiles constraints into code over tracked geometric elements, separating constraints that must hold *during* a subgoal

from those required *on completion* [31]. Others detect failure from the policy's own statistics or a VLM's judgement of progress [14], sometimes closing the loop to recovery [32]. The ancestor of all of this is execution monitoring of high-level robot programs [33], [26].

Three observations place our claim against this cluster, and they are not the ones we expected.

Stage structure is either consumed or produced without a horizon. AHA predicts success *for a given sub-task* from keyframes, and Guardian is conditioned on proposed plans or subtasks and verifies an execution from its before-and-after observations [18], [29]: both *consume* a decomposition someone else supplied. DoReMi and Code-as-Monitor *produce* one, and their during-execution and on-completion constraints correspond closely to our invariant and exit condition [30], [31]. But in the parts of these systems we examined the obligations are *state* constraints with no horizon, and DoReMi's only temporal bound is a global task timeout. Horizons on progress do exist in practice, and the closest is inside the navigation stack we replaced: Nav2's progress checker fails a controller that has not moved a set distance within a set time [34]. That is a progress budget in all but name. What distinguishes ours is where it sits — outside the component it judges, declared per stage in a specification, and producing a categorised verdict rather than the controller's own abort — and we claim that placement, not the mechanism.

So the detectability gap survives the cluster. A robot wedged while still holding its object, with no obstacle newly in front of it, violates no state constraint; and if its stage has not ended there is no boundary at which to notice. Guardian *can* see such a case, and tellingly *no progress* is its best-detected class — but it sees it by comparing a subtask's start and end frames once the subtask is over. We read that as support rather than refutation: stage boundaries are what make progress failure visible, and a per-stage deadline is what lets the verdict arrive *inside* the stage instead of at its end.

The failure data says the same thing. AHA's seven synthesised modes are action-centric — incomplete grasp, slip, keyframe translation and rotation offsets, missing rotation, wrong action sequence, wrong target object — and include no progress or timeout class at all [18]. A literature whose failure datasets contain no stalls will not produce detectors that catch them, which is one reason we think this gap has gone unremarked.

Two smaller points. These monitors return a VLM judgement, so no verdict is monotone and a supervisor has nothing to latch; Section II-C gives our reading of that trade, and architecturally we admit such a detector in the one role where it costs nothing — grounding a single atomic proposition, which we do use. And Code-as-Monitor, having to choose failure thresholds either by hand or from a VLM's prior, states that the effect of threshold variation remains unexplored [31]; that is exactly our H2 (Section VII), which makes bound sensitivity a contribution rather than only a concession.

How to ground propositions in raw perception is itself active [35], [36], and progress there enters our system as a better grounding rule, not as a change to the phase structure.

E. The self-report trap

One finding from the agent literature names the trap our platform fell into. Asked whether language-model agents can report their own task progress, Wang *et al.* find reliability is stage-dependent and collapses mid-task, and conclude a progress report should be checked against independent state rather than treated as control authority [37]. The robotics instance is exact: our navigation stack published `following` while the robot was wedged, and five of the eight propositions in our shipped specification read that self-report. A monitor that must ask the planner whether the planner is stuck is not monitoring the planner. Phase deadlines are the structural answer, because elapsed ticks cannot be misreported.

F. Specification acquisition from language

Translating natural language into temporal logic is a substantial subject [38], [20], [39], some of it with calibrated abstention so an unreliable translation is rejected rather than executed [19]. We take it as given — our generator is an instance — and note only that it translates a *command* into a formula, whereas a monitor needs the richer phase structure authored, which is more constrained and therefore easier to validate mechanically.

G. Position

Monitoring algorithms for LTL are mature, and phase structure is increasingly recognised as necessary — obtained by learning it from labelled episodes [15], [10], by borrowing it from the controller [17], by decomposing formulas for a planner [25], or by having a foundation model emit per-stage constraints [30], [31]. Stage structure is therefore not what we claim as novel. What is missing across all of it is a *horizon*: the obligations attached to a stage are states to maintain, never bounds within which to make progress, so the property class never changes and the stall stays unrefutable. Our claim is that phase structure is not a convenience but a consequence of a property-class fact, and, following from it, a phase structure designed for monitoring: declared rather than learned, carrying deadlines and progress budgets as well as pre- and post-conditions, graded into an intervention, and run alongside LTL so cross-cutting invariants keep a permanent verdict.

IV. Why Phases

A. The detectability gap

Classify the formulas a skill specification actually contains. The cross-cutting invariants, $\mathbf{G}\neg \mathit{collision_risk}$ and $\mathbf{G}\ \mathit{upright}$, are safety properties: a bad prefix exists, they are refutable, and in practice they are the formulas that work. The goal formula (1) is a guarantee property, so under LTL_3 a monitor for it returns $\top$ or ? and never $\bot$ (Section II-B).

The practical reading is blunt. *The formula that states what the skill is for is the formula that can never report the skill failed.* A robot wedged, circling, or standing still with a goal pending produces an unbounded run of INCONCLUSIVE verdicts, and no amount of monitoring effort changes it, because the property admits no finite counterexample. In our

experience this is the dominant robot failure mode, and it is the one a flat specification cannot refute.

Two escapes exist and neither suffices alone. A *terminal failure condition* — our shipped specification has one — fires when the planner reports being stuck, which works only when the planner admits the problem and so returns us to the self-report trap [37]: the wedged humanoid was reporting `following`. *Bounding the goal formula*, writing $\mathbf{F}_{\leq k}$ for $\mathbf{F}$, is the right instinct and is the serious alternative; we take it up in Section IV-F. A third escape, widespread in the foundation-model literature, is to attach state constraints to each stage and check them continuously (Section III-D); it catches a great deal, but not a robot that is stuck while every declared state still holds.

B. Phases as bounded obligations

A phase is a named stage with five guards and two budgets:

- **enter condition** — when the ladder advances into it;
- **precondition** — checked once on entry; a breach says the skill was started in a state it cannot proceed from;
- **invariant** — checked every tick the phase is active;
- **progress condition** with a **violation budget** m: a breach on m consecutive ticks is a progress fault;
- **exit condition** — advances the ladder;
- **deadline** k in ticks: overstaying is a timeout fault.

The deadlines close the gap, and it is worth separating what is proved from what is merely improved. "This phase is eventually left" is a guarantee property with no bad prefix. "This phase is left within k ticks" is decided within $k+1$ positions: it has bad prefixes of length $k+1$, so it is a safety property as well as a guarantee one. Bounding an obligation is therefore what changes its property class — and that is a fact about the bound, not about the decomposition. What the decomposition adds is that the bounds are per stage, that a progress budget has a stage to be scoped to, and that a breach is localised; we separated those claims in Section IV-F and we do not conflate them here.

The consequence for the ladder as a whole is a statement about time to verdict, which is the form in which it is worth proving.

Proposition 1: Let a ladder have phases $1 \dots n$ visited in order, each declaring a finite deadline k_i. Then within $\sum_{i=1}^{n}(k_i+1)$ ticks of *entering the first phase*, the monitor has either completed the ladder or raised a fault.

Proof. The ladder occupies exactly one phase at a time. A phase's elapsed-tick counter is reset on entry and incremented once on every tick the phase remains active, so it measures occupancy of the current phase. On any tick at which the counter has reached k_i the machine raises a timeout fault before it consults the progress or exit conditions, and that fault is terminal. Hence a phase is occupied for at most k_i+1 ticks, the last being the one that faults. A phase is left only forwards, to its successor, and never re-entered, so total occupancy before either leaving the last phase or raising a fault is at most $\sum_i (k_i+1)$. ∎

Two hypotheses are doing work and are stated because the implementation permits their absence. The deadline is optional in the specification format, and a phase without one is bounded by nothing; and the ladder waits in an idle state, without bound, for the first phase's enter condition — so the proposition is about a skill that has started, and says nothing about one that never does.

Two things follow. First, the obligation that *finished* is eventually reached — never refuted on a finite trace when written $\mathbf{F}$ *finished* — is under the ladder *decided* within a horizon fixed by the specification. Second, and less comfortably for the evaluation, detection on a non-succeeding episode is therefore a matter of construction rather than of measurement: we prove it in Proposition 1 and do not claim it as a result (Section VII).

The proposition rests on the deadline alone, and deliberately so: the timeout path counts ticks and evaluates no expression, so no sensor's availability bears on it. The progress budget is the opposite — it is what brings the *typical* detection delay far below that bound, and it does depend on its proposition being evaluable. The deadline gives the guarantee; the budget gives the latency; only the first is proved.

What the episode outcome is a function of, then, is guards that can actually be refuted: a breached invariant, a failed precondition, an exceeded deadline or an exhausted progress budget each end the episode as failure, while the last phase's exit condition ending the ladder is success.

Being precise about what is claimed: we have not made liveness refutable, which is impossible. We have replaced an unbounded obligation with a bounded one — a different and stronger requirement — and the price is that the bounds become part of the specification and must be chosen. We think that is the right price. A deadline is a statement about the skill that whoever describes the skill knows and can state, whereas an unbounded $\mathbf{F}$ is a statement nobody believes: no operator thinks the robot may take arbitrarily long.

C. What else the decomposition buys

Localisation and grading. Every phase fault is named for its stage and guard kind, so the monitor reports that the tracking phase stopped progressing rather than that a conjunction failed — and the phase index is directly the answer to "where in the skill is it", which no product automaton state provides. Each guard also declares a fault category selecting a rung of an intervention ladder: safety breaches halt, timeout and progress breaches request a replan. One consequence is easy to get wrong and worth recording: a phase timeout ends the episode while its category asks for a replan, so the monitor stops stepping exactly while the plan is redone, leaving the robot unmonitored at the worst moment. "This episode is over" and "stop the robot" are different statements and must be reconciled explicitly.

A smaller active guard set. Only the active phase's guards and the propositions the automata currently need are evaluated, and the monitor publishes that set each tick. Since some propositions are grounded by an expensive model rather than a threshold, narrowing the set is what keeps the slow path off the critical path. The same scoping also localises sensor-staleness damage, as Section IV-E describes.

D. Why both layers, and not either alone

Phases without LTL lose the cross-cutting requirements and the monotone verdict: the invariants that matter hold in every phase and between phases, writing them per phase would duplicate them once per stage and lose them during transitions, and a hand-rolled stage machine cannot promise a verdict will not be retracted — which is what a latched safety response depends on. LTL without phases is Section IV-A. So both run on one tick over one proposition assignment: the automata are the verdict of record for cross-cutting properties, the ladder is the bounded, localised, graded account of progress. We did not pick a temporal logic and then add stages for convenience; we needed bounded obligations and a permanent verdict at once, and neither layer provides both.

E. Trusting the answer: three-valued propositions

Guards are evaluated from propositions, propositions are grounded in sensor keys, and sensors go stale. Resolving an unknowable proposition to false is the obvious implementation and is wrong in a specific way: an invariant such as $upright \wedge \neg collision_risk$ then breaches whenever odometry drops a beat, and the monitor manufactures a safety halt out of a transport hiccup.

The design is to make propositions three-valued — TRUE, FALSE, UNKNOWN — and to add a per-tick DECIDED / UNDECIDED status with one rule: if a proposition required by the active phase's guards or by the automaton's current state is UNKNOWN, do not step; record the tick UNDECIDED. The automata stay structurally two-valued, which is deliberate, and freezing preserves that rather than forcing a redesign of the automaton layer.

What ships today is weaker, and we state it rather than describe the design as though it were built. The wire contract already carries an `unknown` list beside the proposition values, so an unevaluable proposition is never encoded as `false`; but the monitor logs that list and steps anyway, and the two layers then treat a missing proposition in *opposite* directions. A phase guard whose expression cannot be evaluated falls back to a pass, so the ladder fails open: an unknown proposition suppresses phase faults. The automata do the reverse: a proposition absent from the assignment is read as false, so $\mathbf{G}\, upright$ is *violated* by a missing $upright$ while $\mathbf{G}\neg collision_risk$ is silently satisfied by a missing $collision_risk$. Neither is the freeze, the first of the two automaton behaviours is exactly the fabricated safety fault this section opened with, and which one a formula gets depends on nothing more principled than the polarity in which its proposition was named. In practice the evaluator holds the last value of a stale key, so propositions go stale far more often than they go missing, and staleness lowers the confidence that de-escalates a fault to a warning; but the gap is real and the freeze is what closes it.

The consequence for Proposition 1 is precise. The timeout path evaluates no expression, so the bound is untouched: a stalled robot with every proposition unknown still produces a timeout fault within the stated horizon. The progress path is suppressed, so the budget stops shortening the delay exactly when sensing degrades. The guarantee survives a sensor outage; the latency does not.

Two points deserve care. First, UNDECIDED is a *different axis* from LTL_3's ?, and the two must not be merged into one enumeration. ? is a claim about the trace and is the normal condition of a healthy obligation; UNDECIDED is a claim about observation — we could not see. The distinction also rules out the tempting move of adopting a four-valued verdict for this purpose: RV-LTL's extra values refine ? along the trace axis (Section II-B) and say nothing about whether the observation was available, so they are not the fourth value this needs. Second, the phase structure scopes the damage: only the active phase's guards can freeze the ladder, so a sensor feeding a later phase's exit condition may be stale throughout an earlier phase at no cost. Flat monitoring has no such scope, because every formula is live at every tick.

The honest consequence of the freeze, once deployed: no safety formula can be violated during a data outage. That is epistemically correct — nothing was observed — and it makes the data-health alarm load-bearing for safety rather than decorative. Every episode verdict therefore carries a coverage count, how many ticks were undecided; an episode with $40\,\%$ undecided ticks folding to "reached goal, full confidence" is not a result.

F. The serious alternative: bounding the flat formula

Writing $\mathbf{F}_{\leq k}$ instead of $\mathbf{F}$, or moving to a metric logic, does restore refutability, and is the objection to answer directly. It also deserves more credit than a single global deadline: bounded operators express per-stage obligations perfectly well, since $\mathbf{G}(armed \wedge \neg planning \rightarrow \mathbf{F}_{\leq 30}\, planning)$ needs no phase machine at all. We therefore do not claim that a bounded encoding cannot localise its deadlines. Three things separate it from the ladder.

A progress budget needs a stage to be scoped to. "The progress condition must not fail on m consecutive ticks *while in this stage*" has a scope, and expressing it requires a proposition that says which stage is active. That proposition is a phase machine under another name — ours is declared and inspectable, whereas in a bounded-LTL encoding it would sit either in the propositions or, as in our own implementation of a debounced progress key, inside the adapter, where it is neither declared nor validated. The decomposition does not go away; it only stops being visible.

Cost grows with the bounds. Bounded operators compile to automata carrying counters, so a per-stage bounded encoding of a long mission grows with k, while the ladder is one integer and a handful of guard evaluations per tick regardless of the deadline.

The stage index is not recoverable. A set of independent bounded formulas gives no single answer to "which stage is this", and no one place in which to grade a breach into an intervention: each formula reports its own violation, and the ordering and mutual exclusion of stages are nowhere stated. Phase decomposition is therefore not syntactic sugar for bounded LTL, but neither is it strictly more expressive —

it is a cheaper, declared and more diagnostic factoring of an obligation that bounded LTL can also express. Because that is an empirical claim rather than a formal one, bounded per-stage LTL is a baseline in Section VII rather than an argument we settle here.

G. What this does not fix

The bounds and thresholds are now specification. A deadline of 120 ticks and a collision threshold of $0.25\,\mathrm{m}$ are calibration, not logic, and a wrong one yields a perfectly verified wrong answer.

Phases impose a sequence. Our ladder is a total order, which fits navigation and does not fit concurrent or re-enterable stages. A partial order would close the detectability gap too and is the natural generalisation. Learned monitors meet the same limit from the other side: RoboMonitor notes one global phase per timestep cannot express two arms doing different things [15].

Deadlines in ticks are only as meaningful as the tick. Section V is not an implementation detail of this section; it is its precondition.

V. Trace Semantics

[TODO: Full section, roughly one column plus the tick-window figure, compressed from docs/clocking.md. Points that must survive, now that deadlines and progress budgets are counted in ticks:]

- Three rates named and never conflated: observation rate (which *is* the trace), transport rate, publication rate. Only the first may affect a verdict — so a deadline of k ticks means the same thing live and on replay.
- Tick k is the half-open interval $(B_{k-1}, B_k]$, $B_k = t_0 + k\Delta$, $\Delta = 1/tick_hz$, named by the closing boundary. Half-open so every instant belongs to exactly one tick. This is the position index of Section II-A.
- Close is atomic: fold the window per declared per-key policy, commit, run tick-steps, emit observation, step automata once, evaluate phase guards, emit verdict, open the next window.
- A tick fires even when nothing arrived — that is the tick that must report insufficient data. No catch-up merging.
- **Why this section is load-bearing for the phase claim:** a debounce or deadline counted in *messages* rather than ticks means something different on every machine. Our own progress debounce has this bug — 10 ticks documented, once per message implemented, so $2\,\mathrm{s}$ on a $5\,\mathrm{Hz}$ topic instead of $10\,\mathrm{s}$, and in simulation, where the source publishes only on transitions, it never fires at all. A phase budget is only a specification if the tick is defined.
- Clock modes: wall, replay (replay speed cannot change the tick count, hence cannot change a deadline), manual single-step.

VI. How the Monitor Works

A. Artifacts, and who authors each

Four artifacts, deliberately separated by author rather than by module. A **sensor schema** declares, per robot family, the key names a specification may mention and what each means; an integrator writes it. An **adapter descriptor** says which topic and field feeds which key, at what nominal rate and staleness tolerance, and how several samples inside one tick fold to one value; an integrator writes one per embodiment. A **skill specification** carries the atomic propositions with their grounding rules, the cross-cutting LTL formulas, the named failure modes with categories, and the phase ladder; it is generated from a description (Section II-D) or written by a domain expert, per skill. The **grounding rules** live inside the specification. **[TODO: one figure: the shipped G1 specification, annotated with the four artifact boundaries.]**

B. One tick, end to end

The monitor is driven by an external clock, not by message arrival (Section V). On each pulse, atomically:

1) **Fold.** Each key's samples from the closing window reduce to one value by that key's declared policy. A key with no sample holds its previous value, and a per-source freshness record says whether it is stale rather than steady.
2) **Ground.** The propositions the monitor asked for are evaluated against the folded values. A rule of the form "True when ⟨expression⟩" is evaluated directly in a restricted environment, in well under a millisecond. A proposition whose description is not reducible to a threshold goes to a model on a worker thread; the queue is drained when a newer request arrives, so a late answer is discarded rather than applied to the wrong tick. Stale sources are reported beside the values and lower a confidence scalar that travels with them.
3) **Account for what could not be evaluated.** A proposition that could not be evaluated is listed separately rather than encoded as false. As built, the monitor logs that list and steps regardless; the freeze of Section IV-E is designed and not deployed.
4) **Step the automata.** One BDD evaluation and one state assignment per cross-cutting formula. A formula entering its trap state is VIOLATED, permanently.
5) **Step the ladder.** The active phase's guards are consulted in a fixed order — *precondition*, once, on entry; then on every tick the *invariant*; then the *deadline*, by comparing the phase's elapsed-tick count with its bound; then the *progress condition*, incrementing or resetting its violation counter; then the *exit condition*, which advances the ladder if the next phase's enter condition also holds. The deadline is checked before any condition that depends on a proposition other than the invariant's, which is what Proposition 1 relies on.
6) **Publish.** One verdict carrying the active phase and index, each formula's status, the evaluated truth of every guard the phase declares, the violation counters, any fault raised with its name and category, the undecided flag, and the episode's terminal value.

Publishing the guards' evaluated truth matters more than it looks: it is what makes the phase state *inspectable* rather than

inferred, so an operator or a log reader can see which guard was about to break without re-deriving it from sensor values.

C. Both layers, one proposition assignment

The automata and the ladder consume the same assignment on the same tick. The automata own the cross-cutting verdict and contribute permanence; the ladder owns progress and contributes bounds, names and categories. They can disagree — a phase timeout ends the episode while its category asks for a replan — and the disagreement is reconciled at the supervisor rather than inside the monitor, because "this episode is over" and "stop the robot" are different statements and collapsing them inside the monitor loses one of them.

D. From a fault to an intervention

A monitor by itself changes no outcome. The path from detection to a better outcome has four links, and the design makes each explicit.

A guard breach produces a **named fault**, carrying the stage and the guard kind. The fault's **category** is fixed for two guard kinds and declared for the other two: an exceeded deadline is always TIMEOUT and an exhausted progress budget always PROGRESS, while the specification declares the category of each phase's invariant and precondition — in our navigation specification every invariant is SAFETY. The category, together with how imminent the fault is and how confident the monitor is in the signal behind it, selects a rung of a six-step **intervention ladder** — continue, warn, slow, replan, halt, abort — graded in exactly one place so that the node actuating the robot contains no policy. A confirmed safety fault halts or aborts; a confirmed timeout or stall requests a replan; and a fault resting on low-confidence sensing is de-escalated to a warning rather than acted on. The **supervisor** overrides the base with a zero-velocity command while a halting fault is active, and enforces nothing until enabled, which is what makes a detection-only arm available for the evaluation below. Extending the override to every episode end, so that a robot is never left driving while its monitor has stopped stepping, is specified and not yet built.

So the causal claim the paper can make about success rate is narrow and worth stating as such: earlier detection helps only where something useful can be done earlier. For a stall, a replan issued at the progress budget rather than at the mission timeout leaves more of the episode's remaining budget to find another route, and spends fewer ticks driving a wedged robot. Where a goal is genuinely unreachable, no intervention restores success and the gain appears only as execution not wasted — which is why Section VII measures both and does not rest on success rate alone.

E. Nothing here names an embodiment

The monitor's inputs are a specification and an observation stream, and neither mentions a topic, a sensor or a robot. Embodiment-specific knowledge terminates at the evaluator that owns the descriptor; the two halves learn of each other only through latched manifests on the wire, which is what lets a monitor run with no adapter present at all — for instance against a recorded stream. Schema fragments keep this from collapsing into a navigation-only vocabulary: a descriptor composes an embodiment fragment with a task-family fragment, so a manipulation skill keeps the former and drops the latter rather than having to adopt navigation keys before it can say anything.

VII. Evaluation

[TODO: Protocols below are fixed; no numbers are collected yet. H1–H3 run offline on one recorded corpus.]

A. Platform and task

[TODO: Unitree G1 humanoid, onboard perception, the TRAV traversability navigation stack, waypoint missions in [TODO: environment]. Simulation comparisons use MuJoCo [40] and Isaac Lab [41]; the superseded baseline the simulation descriptors were written against was Nav2 [34]. Built on ROS 2 [42].]

B. What we measure, and why

The monitor is a *sequential detector*: each tick it emits continue, or stop with a named fault. A faulted episode has an onset tick ν.

How ν is defined matters more than it appears, and we define it from signals the monitor never reads. The tempting definition — the first tick at which closing speed on the goal falls below a threshold and does not recover — is circular, because our progress condition is itself computed from closing speed, so the detector would be scored against a label derived from its own input and its measured latency would be close to tautological. Instead ν is the tick at which the induced disturbance was applied, where failures are induced; and otherwise it is annotated from the recorded video and an external pose estimate, neither of which appears in any grounding rule. The annotation protocol and inter-annotator agreement are reported with the results. **[TODO: protocol, agreement]**

Two quantities follow, and neither is meaningful alone:

- the **false-alarm rate** $\hat{\alpha}$ — the fraction of *nominal* episodes in which any fault is raised;
- the **detection delay** $d = t_{\text{detect}} - \nu$ in ticks on faulted episodes, together with the **detection rate** ρ, the fraction of faulted episodes detected at all.

Primary metric. *Median and worst-case detection delay, at the tightest (k, m) whose false-alarm rate on a held-out nominal set satisfies $\hat{\alpha} \leq \alpha_0$.*

Detection rate is reported but is not the result, and it is worth saying why before the numbers rather than after. Proposition 1 already establishes $\rho = 1$ for the ladder on any non-succeeding episode, and Section IV-A establishes $\rho = 0$ for B1 on progress failures at every budget. Both are theorems about encodings, not measurements. We report ρ as a *conformance check* — evidence that the implementation honours the proposition, which is a claim about our code and not about the idea — and we rest the empirical argument on what the proposition does *not* settle: how far below the worst-case bound the delay actually falls, and what that costs in false

alarms. That is the honest division, and it is why Section VII-D rather than Section VII-C carries the paper's empirical weight.

Four reasons this is the right metric rather than one we preferred.

It is the established criterion for this problem class. A phase deadline and a progress budget are a stopping rule, so the monitor is a sequential change detector, and such rules have been evaluated since Page's CUSUM by a pair of run lengths — to false alarm, and to detection [43]. Lorden made the pairing precise: minimise worst-case expected detection delay subject to a false-alarm constraint, a criterion under which CUSUM is minimax optimal [44], [45]. We adopt that criterion instead of inventing one. Two adaptations, stated because the correspondence is close but not exact: our episodes are finite, so the finite-horizon analogue of a run length to false alarm is a per-episode false-alarm *probability* rather than an expected time, and we measure that probability directly rather than approximating it as a reciprocal; and our detector is not derived from a likelihood ratio, so we claim no optimality — only that this is the criterion the problem class already has.

Neither half is gameable alone. Detection rate is maximised by $k = m = 1$, which detects every failure and aborts every healthy run; delay is maximised the same way. Constraining one while reporting the other cannot be gamed, and any paper reporting detection without a false-alarm budget has reported nothing.

The bound sweep is the operating characteristic, not a side experiment. Sweeping (k, m) traces delay against false-alarm rate, per failure class. The headline number is a point on that curve chosen by a stated budget — which is also the answer to the sharpest objection available, that the bounds were tuned to the failure set: we report the curve rather than a point.

It states the baseline's failure correctly rather than unfairly. The flat encoding's curve on progress failures is *empty*: $\rho = 0$ at every false-alarm budget, because no setting of any parameter can give a guarantee property a bad prefix. That is a property of the encoding, not of a tuning choice, and it is the strongest available statement of the paper's claim.

Six commitments made before any faulted episode was examined, each because there is a way to get it wrong. **The budget α_0 is fixed in advance**, not chosen once the curves are in hand; selecting it afterwards would make the headline a best-of-many report, and we additionally publish the full curve so the choice is auditable. **Delay is right-censored**: undetected episodes have no finite d, so we report ρ separately and the median d over detected episodes, never a mean that silently drops non-detections — which would flatter whichever arm detects less often but faster. **Stratify by failure class**: a topple trips an invariant on the first tick under both encodings, so pooling it with a stall dilutes precisely the effect under test. **Gate on coverage**: an episode whose undecided fraction exceeds a stated bound is excluded and counted, or a sensor dropout is reported as a monitoring result (Section IV-E). **Establish the nominal set by outcome and by review**: an episode counts as nominal only if it both terminated successfully and survived manual inspection, and we report how many candidates review rejected — an undetected sub-failure surviving into the nominal set inflates the measured false-alarm rate, which biases against our own claim rather than for it. **Report delay in both units**: a budget of k ticks is a different wall-clock duration at a different tick rate, so $tick_hz$ is held identical across arms, stated with the results, and every delay is given in ticks and seconds.

Why not AUROC, which much of the neighbouring literature reports: it summarises a frame-wise classification and is threshold-free, whereas this is a stopping problem in which *when* is the entire question. A detector with perfect AUROC that fires on an episode's last tick is worthless, and AUROC cannot tell it from one that fires at onset. The informative part of that summary here is per-class detection rate, which we do report.

C. H1: conformance, delay and localisation

Over one corpus of recorded episodes we replay *four* encodings of the same mission. All four see an identical observation stream and share the same propositions and cross-cutting invariants, so the comparison isolates the encoding alone.

- **B1 — flat.** Cross-cutting invariants plus the nested-**F** goal formula. Establishes the formal point of Section IV-A: $\rho = 0$ on progress failures at every budget.
- **B2 — flat with a mission deadline.** The same, plus one global timeout. This is what a competent engineer builds without reading this paper, and it is the baseline that keeps B1 from being a straw man.
- **B3 — per-stage bounded LTL.** Bounded-**F** obligations per stage, with no phase machine — the strongest formal alternative, and the one Section IV-F declines to settle by argument.
- **C — the phase ladder.**

[TODO: Report ρ and median/worst-case d per failure class (wedged, dead end, circling at constant distance, sensor dropout, toppling) at the pre-registered α_0. The informative comparisons are C against B2 on delay and localisation, and C against B3 on cost and inspectability — if C beat B3 on detection we would expect an implementation fault in one of them rather than a result, and we say so in advance. Report separately the episodes in which the planner's own self-report fired, since those flatter B1 and B2 for the wrong reason (Section IV-A).]

D. H2 (main): the operating characteristic

This is a headline result rather than an ablation, and Figure 2 carries it. Sweeping the deadline k and the progress budget m per phase over the same corpus traces detection delay against false-alarm rate, per failure class; H1 reports one point on these curves, at the pre-registered budget. Reporting the curve is what answers the two objections the design most invites — that the bounds were tuned to the failure set, and that adding a deadline is definitionally sufficient to detect timeouts — because the curve shows what the deadline *costs* as well as what it buys. It is also the measurement a close competitor states is missing, having to choose failure thresholds by hand or from a model's prior with the effect of variation left unexplored [31].

[TODO: Figure: detection delay against per-episode false-alarm rate, one curve per failure class, points labelled by (k, m). Mark the pre-registered α_0 with a vertical line and the reported operating point on each curve. Overlay B2 and B3 as single points or their own curves. B1 appears as a flat line at $\rho = 0$, which is the visual statement of Section IV-A.]

Fig. 2. The operating characteristic of the phase ladder: what the bounds cost in false alarms, against what they buy in detection delay. H1's headline is one point on this family of curves.

E. H3: execution not wasted

[TODO: Wasted ticks = episode end $-\nu$ on faulted episodes; the monitor's saving is the difference against the unmonitored arm. This is the utility measure that does not require recovery to succeed, which matters because an unreachable goal admits no intervention and a success-rate-only evaluation would score a perfect monitor at zero. DoReMi reports execution time for the same reason [30].]

[TODO: Also report here, once per corpus rather than per hypothesis: the failure-mode distribution of *induced* against *naturally occurring* failures, as Guardian does for synthetic against real failures [29], so a reader can judge how far the induced set stands in for the real one; and a provenance table for the four grounding thresholds, giving for each whether it was measured from this corpus or chosen, since every curve above is conditional on their values.]

F. H4: does it raise task success rate?

Four arms, with the intervention policy and the recovery repertoire held fixed so that only the monitor varies: (A) no monitor; (B) flat specification; (C) phase-structured specification; (D) an oracle monitor fired at ground-truth onset. The claim is $C > B$. Arm D bounds what any monitor could achieve with this recovery repertoire, so the result can be reported as a fraction of achievable gain rather than an unanchored percentage. The ladder follows DoReMi's use of an oracle upper bound [30], and we report false interventions alongside successes as RoboMonitor does [15]. **[TODO: numbers]** Stated plainly: success rate is a joint property of monitor, intervention policy and recovery repertoire, and only holding the latter two fixed makes the comparison about the first.

One pre-check decides whether this hypothesis is worth running at all. If a replan issued at the moment of a stall rarely recovers on this platform, arms B, C and D all collapse onto A, and the null result would say something about the recovery repertoire rather than about the monitor. We therefore measure recovery success on a small set of induced stalls first, and if it is low we demote H4 and lead on H3, where the monitor's contribution does not depend on recovery succeeding. **[TODO: pre-check result]**

G. H5: cost and coverage

[TODO: Per-tick cost split into BDD steps, guard evaluation, fast-path grounding, and slow-path hit rate with and without active-guard narrowing; plus the undecided fraction, since every result above is conditioned on coverage.]

H. H6, H7: the two consequences

[TODO: H6 (portability): one recorded episode through two independent paths — re-execution in simulation through a different descriptor, and replay of the recorded observation stream — must produce the same verdict, compared over the full record with nothing ignored by default. H7 (authorship): generate specifications for n skill descriptions, report generate/validate/repair outcome rates against the two gates of Section II-D, and test whether a generated ladder detects the H1 failure classes as well as the hand-authored one — the claim the hand-authored reference cannot support. H7 also carries the only evidence we offer for skill-agnosticism beyond construction: a second, non-navigation skill exercised *at specification level only* — generated from a description, validated against its schema, and shown to compose the shared embodiment fragment without adopting navigation keys. That needs no robot and no second corpus, and it tests the authorship and vocabulary claims while deliberately not claiming a second monitoring result (Section VIII).]

VIII. Threats to Validity

[TODO: Written against the planned corpus. Re-check every item below against the corpus actually collected before submission — several mitigations are commitments that must have been kept.]

A. To the claim

Stage structure is prior art; we claim only the horizon. Decomposing execution into stages is widespread — learned from labelled episodes [15], [10], borrowed from the controller [17], or emitted per skill by a language model [30], [31] — and we do not claim it. What we claim is the bound attached to a stage, and the property-class consequence of attaching it. A reader who regards a behaviour tree's timeout decorator or a global task timeout as already constituting a per-stage horizon will find the contribution narrow; our reply is independence and categorisation, and arms B2 and B3 of Section VII are where that reply is tested rather than asserted.

Bounded LTL can express per-stage deadlines, and we do not claim otherwise. Section IV-F concedes this and reduces our position to three empirical differences: cost, declaredness, and the recoverability of a stage index. Arm B3 measures them. We state the unfavourable outcome in advance so that it cannot be reported as a surprise: if B3 matches the ladder on detection rate and delay, the contribution reduces to those three differences, which are real but smaller than the paper's framing suggests.

The flat baseline is not an engineering baseline. B1 exists to exhibit a property-class fact, not to stand for practice; nobody deploys a bare nested-**F** formula with no timeout. B2 is the baseline that carries the comparison, and B1's result should be read as a statement about encodings rather than as a defeat of anyone's system.

B. To the measurement

Freeze-on-unknown is designed and not deployed. Section IV-E describes a three-valued proposition layer with a freeze rule; what ships carries the unknown list on the wire but steps anyway, with phase guards falling back to a pass and the automata reading a missing proposition as false — so a missing proposition can both hide a phase fault and fabricate a safety one. We describe both rather than the design alone, because a paper that misstates its own system's behaviour is worse than one with a gap. Residual: Proposition 1 is unaffected, since the timeout path evaluates nothing, but the progress budget stops shortening delay when sensing degrades — so measured latency under poor coverage reflects the deadline rather than the budget, and the coverage figure must be read alongside every delay we report.

Detection is proved, not measured, and that cuts both ways. $\rho = 1$ for the ladder and $\rho = 0$ for B1 follow from the encodings (Proposition 1, Section IV-A). A reader expecting an empirical detection result will not find one, and should not: the measurable question is cost, which is H2. Residual: a reviewer may regard a proved detection claim as weaker evidence than a measured one, on the grounds that the proposition is a statement about the specification rather than about the robot. That is correct, and the conformance check in H1 plus the latency and false-alarm figures are what connect the proposition to the artifact.

Planner self-report inside the specification. Five of the eight propositions in the shipped navigation specification read the navigation stack's own status — which is the trap Section IV-A names, present in our own artifact. The planner-independent redesign derives progress from odometry against the commanded goal and forbids every planner topic by test. It must land before the headline corpus is recorded; episodes in which the planner's self-report fired are reported as a separate stratum either way. Residual: until it lands, the independence half of our argument is architectural rather than demonstrated, and any detection B1 or B2 shows on a stall is partly the planner's own admission rather than the monitor's work.

Induced failures need not match the real distribution. We perturb executions to obtain failures at useful density, which is the standard criticism of synthetic failure data [18], [29]. Mitigation: we report the failure-mode distribution of induced against naturally occurring failures, as Guardian does, and report natural failures as their own stratum. Residual: the natural sample is small, so the comparison bounds the concern rather than removing it.

Grounding thresholds are calibration, not logic. Four of them — arrival radius, the closing-speed epsilon, the no-progress debounce and the minimum-range height band — are measured from the same recording that feeds H1, and we report each threshold's provenance. Residual: a wrong threshold yields a perfectly verified wrong answer, and the operating characteristic of Section VII-B shifts with it, so every curve we report is conditional on the calibrated values.

A debounce counted in messages rather than ticks. The shipped progress debounce advances once per incoming message, so the same specification means $2\,\mathrm{s}$ on a $5\,\mathrm{Hz}$ topic where it documents $10\,\mathrm{s}$, and in simulation, where the source publishes only on transitions, it never fires at all. It is corrected as one identifiable commit before any collection. Residual: the fix turns on a signal that has never been on, so no simulation figure predating it is comparable and we publish none.

Coverage gating is itself a selection. Episodes whose undecided fraction exceeds a stated bound are excluded, because a sensor dropout reported as a detection result would be worse (Section IV-E). Residual: gating selects, so we report the exclusion count alongside every result and probe the undecided fraction early in collection rather than discovering it afterwards.

C. To the metric

Onset definition. Scoring detection delay against a label derived from the detector's own input would be circular, so ν comes from the applied disturbance or from video and an external pose estimate, neither of which appears in any grounding rule (Section VII-B). Residual: annotating when a stall *began* is a judgement; we report the protocol and inter-annotator agreement, and the delay figures should be read with that agreement in mind.

Choice of false-alarm budget. α_0 is fixed before any faulted episode is examined and the full delay-against-false-alarm curve is published, so a reader can see the headline's position on it rather than taking the operating point on trust.

Purity of the nominal set. Nominal episodes are selected by outcome and by manual review, and we report how many candidates were rejected. Residual: an undetected sub-failure surviving review inflates the measured false-alarm rate — which biases *against* our claim, so the direction of this error is the safe one.

Delay is denominated in ticks. A deadline of k ticks is a different wall-clock duration at a different tick rate, so $tick_hz$ is held identical across arms, stated, and all delays are reported in both ticks and seconds.

Success rate is a joint property. H4 varies only the monitor, holding the intervention policy and recovery repertoire fixed, and includes an oracle arm so the result can be expressed as a fraction of achievable gain. Residual: if replanning rarely recovers a stall on this platform, all monitored arms collapse onto the unmonitored one and the null result would describe the recovery repertoire rather than the monitor — which is why a recovery pre-check gates the hypothesis and why H3 does not depend on recovery succeeding.

D. To generalisation

One skill family executed. Navigation is the only family run end to end on hardware. A second, non-navigation skill is exercised at specification level only — generated, validated against its schema, and shown to compose the shared embodiment fragment without adopting navigation keys — which tests the authorship and schema claims but not the monitoring result. Residual: no second family is executed, so skill-agnosticism remains supported by construction and by schema parity rather than by a second measurement.

The reference specification is hand-authored. It is the evaluation reference, not the generator's output, and H7 is run on generated specifications only. Presenting the hand-authored specification as a product of the pipeline would falsify the authorship claim outright.

The ladder is a total order. It fits a waypoint mission and cannot express concurrent or re-enterable stages; a partial order would close the detectability gap equally and is the natural generalisation. Learned monitors meet the same limit from the other side, one global phase per timestep being unable to describe two arms doing different things [15].

One robot, one navigation stack. Results are from a single G1 with a single traversability planner. The portability argument (Section VI) is what makes us expect transfer, and H6 is the only evidence offered for it.

No authentication anywhere in the stack. The deployment model is a trusted robot tier behind a terminating proxy, stated here rather than left implied.

IX. Conclusion

[TODO: Write once Section VII has numbers. The shape: the formula that says what a skill is for is a guarantee property with no bad prefix, so it can never report the skill failed, and a flat temporal specification is blind to the commonest robot failure. Decomposing execution into phases with deadlines and progress budgets converts that unrefutable obligation into a ladder of refutable ones, localises a breach to a stage and a guard, and grades it into a response. LTL is not displaced but required alongside, for the cross-cutting invariants and the permanent verdict phases cannot promise; and because a deadline is what closes an episode, the phase structure supplies the finiteness a finite-trace semantics assumes. Portability and language authorship follow from the same decomposition. Next: partial-order phases, odometry-derived progress, STL guards for margin-shaped requirements.]

References

[1] Z. Manna and A. Pnueli, "A hierarchy of temporal properties," in *Proc. 9th ACM Symposium on Principles of Distributed Computing (PODC)*, 1990, pp. 377–410.

[2] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, 2011.

[3] A. Pnueli, "The temporal logic of programs," in *Proc. 18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.

[4] M. Y. Vardi and P. Wolper, "An automata-theoretic approach to automatic program verification," in *Proc. 1st IEEE Symposium on Logic in Computer Science (LICS)*, 1986, pp. 332–344.

[5] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, É. Renault, and L. Xu, "Spot 2.0 — a framework for LTL and ω-automata manipulation," in *Proc. 14th International Symposium on Automated Technology for Verification and Analysis (ATVA)*, ser. LNCS, vol. 9938. Springer, 2016, pp. 122–129.

[6] G. D. Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proc. 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2013, pp. 854–860.

[7] A. Bauer, M. Leucker, and C. Schallhart, "Comparing LTL semantics for runtime verification," *Journal of Logic and Computation*, vol. 20, no. 3, pp. 651–674, 2010.

[8] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.

[9] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," *Robotics and Autonomous Systems*, vol. 154, p. 104096, 2022.

[10] D. Sliwowski and D. Lee, "ConditionNET: Learning preconditions and effects for execution monitoring," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1337–1344, 2025.

[11] R. T. Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Reward machines: Exploiting reward function structure in reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022.

[12] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals," in *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT)*, ser. LNCS, vol. 3253. Springer, 2004, pp. 152–166.

[13] G. E. Fainekos and G. J. Pappas, "Robustness of temporal logic specifications for continuous-time signals," *Theoretical Computer Science*, vol. 410, no. 42, pp. 4262–4291, 2009.

[14] C. Agia, R. Sinha, J. Yang, Z. ang Cao, R. Antonova, M. Pavone, and J. Bohg, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 689–723.

[15] A. Ajith, G. Narayanan, K. Coelho, T. Zhao, Y. Shahapurkar, B. Zhu, M. Erdogan, T. Krubasik, C. Chamzas, and E. Solowjow, "RoboMonitor: Label-efficient runtime monitoring of robot task execution via predictive representation learning," *arXiv preprint arXiv:2609.30715*, 2026.

[16] F. Ahmad, H. Ismail, J. Styrud, M. Stenmark, and V. Krueger, "A unified framework for real-time failure handling in robotics using vision-language models, reactive planner and behavior trees," in *Proc. IEEE 21st International Conference on Automation Science and Engineering (CASE)*, 2025, pp. 887–894.

[17] M. Colledanchise, G. Cicala, D. E. Domenichelli, L. Natale, and A. Tacchella, "Formalizing the execution context of behavior trees for runtime verification of deliberative policies," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021.

[18] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[19] D. S. Sundarsingh, J. Wang, J. V. Deshmukh, and Y. Kantaros, "ConformalNL2LTL: Translating natural language instructions into temporal logic formulas with conformal correctness guarantees," *arXiv preprint arXiv:2504.21022*, 2025.

[20] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, "NL2TL: Transforming natural languages to temporal logics using large language models," *arXiv preprint arXiv:2305.07766*, 2023.

[21] M. Leucker and C. Schallhart, "A brief account of runtime verification," *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[22] Y. Falcone, S. Krstić, G. Reger, and D. Traytel, "A taxonomy for classifying runtime verification tools," *International Journal on Software Tools for Technology Transfer*, vol. 23, no. 2, pp. 255–284, 2021.

[23] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, "Temporal-logic-based reactive mission and motion planning," *IEEE Transactions on Robotics*, vol. 25, no. 6, pp. 1370–1381, 2009.

[24] H. Kress-Gazit, M. Lahijanian, and V. Raman, "Synthesis for robots: Guarantees and feedback for robot behavior," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, no. 1, pp. 211–236, 2018.

[25] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "Nl2Hltl2Plan: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task representation," *arXiv preprint arXiv:2408.08188*, 2024.

[26] O. Pettersson, "Execution monitoring in robotics: A survey," *Robotics and Autonomous Systems*, vol. 53, no. 2, pp. 73–88, 2005.

[27] Z. Shi, W. Zhang, X. Jiang, X. Wu, C. Yang, C. Xu, J. Guo, and Y. Wang, "DynSTEER: Dynamic stage-wise trajectory evaluation and execution-time review for agents," *arXiv preprint arXiv:2609.14637*, 2026.

[28] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 229, 2023, pp. 3468–3484.

[29] P. Pacaud, R. Garcia, S. Chen, and C. Schmid, "Guardian: Detecting robotic manipulation failures with vision-language models," *arXiv preprint arXiv:2512.01946*, 2025.

[30] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen, "DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[31] E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang, "Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection," *arXiv preprint arXiv:2412.04455*, 2024.

[32] A. V. Periasami, X. Luo, H. Li, and X. Cheng, "FRAMES: Failure recovery and monitoring of embodied skills for humanoid loco-manipulation," *arXiv preprint arXiv:2609.22538*, 2026.

[33] G. D. Giacomo, R. Reiter, and M. Soutchanski, "Execution monitoring of high-level robot programs," in *Proc. International Conference on Principles of Knowledge Representation and Reasoning (KR)*, 1998.

[34] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[35] A. Balakrishnan, J. Deshmukh, B. Hoxha, T. Yamaguchi, and G. Fainekos, "PerceMon: Online monitoring for perception systems," in *Proc. International Conference on Runtime Verification (RV)*, 2021.

[36] P. Kapoor, A. Hammer, A. Kapoor, K. Leung, and E. Kang, "Runtime monitoring of perception-based autonomous systems via embedding temporal logic," *arXiv preprint arXiv:2605.12651*, 2026.

[37] B. Wang, Y. Wang, and Y. Wu, "The unreliable progress bar: Can LLM agents reliably report task progress throughout execution?" *arXiv preprint arXiv:2609.08589*, 2026.

[38] J. X. Liu *et al.*, "Grounding complex natural language commands for temporal tasks in unseen environments," in *Proc. Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 229, 2023.

[39] F. Fuggitti and T. Chakraborti, "NL2LTL — a python package for converting natural language instructions to linear temporal logic formulas," in *Proc. AAAI Conference on Artificial Intelligence (Demonstration Track)*, 2023.

[40] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[41] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[42] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[43] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[44] G. Lorden, "Procedures for reacting to a change in distribution," *The Annals of Mathematical Statistics*, vol. 42, no. 6, pp. 1897–1908, 1971.

[45] G. V. Moustakides, "Optimal stopping times for detecting changes in distributions," *The Annals of Statistics*, vol. 14, no. 4, pp. 1379–1387, 1986.